

FÁBRICA DE SITES AI

Documentação Executiva Completa

Auditoria técnica e leitura factual do produto:
arquitetura, regras de negócio, fluxos, KPIs, riscos,
roadmap, plano de melhoria orientado por dados e
decisões estratégicas.

Data da auditoria: 24 de julho de 2026

Base: leitura direta do repositório (código + documentação)

Documentos incluídos: 9

Sumário

- 1.01 — Resumo Executivo
- 2.02 — Arquitetura
- 3.03 — Regras de Negócio Implementadas
- 4.04 — Fluxo do Negócio e Modelo Comercial
- 5.05 — KPIs
- 6.06 — Roadmap Executivo
- 7.07 — Riscos
- 8.08 — Dados, Melhoria Orientada por Dados e Experimentos
- 9.10 — Decisões Estratégicas do Projeto

01 — Resumo Executivo

Base factual: auditoria técnica completa somente-leitura do repositório inteiro, realizada em 2026-07-24 (backend/, frontend, infra/, docs/, histórico de commits). Toda afirmação abaixo tem origem rastreável em código ou documentação real — nada foi suposto. Onde a informação é interpretação/estratégia (não um fato de código), está marcado como tal.

O que é a empresa, hoje

A **Fábrica de Sites AI** é uma operação de duas partes que já funcionam de verdade, mais uma terceira em esqueleto:

- 1. **Um motor de geração de sites via IA** (backend/agent_construtor.py) que recebe dados de um negócio e devolve um site completo (textos, estrutura, imagens, SEO) em minutos — **congelado por decisão** (estável, sem refatoração sem necessidade explícita, ver CLAUDE.md).
- 2. **Um sistema de prospecção e CRM manual** (backend/routers/hunter.py , "Hunter Online") que busca empresas sem site via Google Places, guarda tudo em Postgres, e dá um pipeline de status (pendente → contatado → respondeu → demo_enviada → cliente → descartado) pra um humano trabalhar os leads manualmente — com dois templates de mensagem prontos pra copiar/colar, nunca enviados automaticamente.
- 3. **Três agentes especializados** (backend/agents/hunter.py , vendedor.py , financeiro.py) com lógica de negócio real (scoring, conciliação de pagamento, geração de mensagem), mas com **toda integração externa mockada** — nenhum dispara WhatsApp de verdade, nenhum processa pagamento de verdade ainda.

Estado geral, por fase do ROADMAP.md :

FASE	O QUE É	ESTADO REAL
Fase 0 — Fundação	Nicho + região definidos, portfólio semente gerado	✅ Concluída
Fase 1 — MVP técnico	Motor de geração + persistência + deploy	🟢 ~85% — falta só atualizar a documentação (o backend já está em produção, só que numa infra diferente da planejada — ver seção "Inconsistências" abaixo) e a cobrança recorrente
Fase 2 — Vendas manuais	Fechar 15-20 clientes usando o Hunter pra prospectar	🟡 Em andamento — infraestrutura pronta, gargalo é comercial, não técnico
Fase 3 — Automação por agentes	Plugar integrações reais nos 3 agentes	🟡 Esqueleto pronto, 0% das integrações externas reais

FASE	O QUE É	ESTADO REAL
Fase 4 — Escala	Dashboard de negócio, métricas, crescimento	● Não iniciada

O achado mais importante desta auditoria

A capacidade técnica está à frente da capacidade comercial. O Hunter consegue prospectar em volume (uma busca no Google Places já traz dezenas de leads por execução, `backend/scripts/buscar_leads_google_maps.py`), o motor de geração cria uma demo em minutos, e a infraestrutura (Postgres + Docker + Zero Trust) aguenta multiplicar esse volume sem mudança de código. O que ainda não existe é o *throughput* comercial — não há, em nenhum lugar do repositório (CSV, banco, documento), um número real de vendas fechadas até hoje. O gargalo do negócio, hoje, é humano: quantas conversas de WhatsApp por dia a operação consegue sustentar, não quantos leads o sistema consegue encontrar.

Riscos que mais merecem atenção do sócio

1. **CORS aberto** (`allow_origins=["*"]` , `backend/app.py:89`) e **ausência de rate limit** em qualquer endpoint — hoje não é um problema prático (baixo tráfego), mas é dívida que cresce em risco proporcionalmente ao sucesso comercial.
2. **Nenhum backup de banco documentado** — hoje toda a base de leads/sites vive em um único Postgres na VM Oracle, sem rotina de backup identificada no repositório.
3. **Documentação central desatualizada**: `CLAUDE.md` e `ROADMAP.md` ainda descrevem uma decisão de infraestrutura (Render/Railway) que foi substituída há tempo pela VM Oracle Zero Trust — risco de alguém (humano ou IA) tomar decisão futura baseada nessa informação errada.
4. **Guardrails comerciais críticos estão sendo respeitados hoje** (nunca WhatsApp automático, nunca depoimento fabricado, nunca venda automatizada antes do gate de 15-20 clientes) — o risco aqui não é técnico, é de disciplina: são regras que dependem de continuar sendo seguidas conforme o código evolui.

(Detalhe completo de cada risco em [07-riscos.md](#).)

Próximos passos priorizados (visão técnica)

1. Corrigir a `DATABASE_URL` do ambiente local (aponta pra uma VM Oracle que já foi desligada, ver inconsistência #1 no relatório de auditoria).
2. Atualizar `CLAUDE.md` / `ROADMAP.md` pra refletir a infraestrutura real (Zero Trust Oracle, não Render/Railway).
3. Restringir CORS e adicionar rate limit básico nos endpoints protegidos por API key.

4. Arquivar/remover a documentação obsoleta da raiz (8 arquivos `.md` de uma versão anterior do produto, pré-reorganização em `backend/`).

(Esta é o backlog técnico. O backlog comercial — o que realmente destrava crescimento — está em [06-roadmap-executivo.md](#) e nasce da constatação da seção anterior: o gargalo é vender, não construir.)

Modelo comercial, em uma frase

(Esta seção é síntese estratégica, não um fato extraído de código — a origem dos dados é `vendas-config.json` para posicionamento/preço e a memória de decisões de negócio do projeto.)

Vendemos um site profissional gerado por IA para pequenos negócios locais sem presença digital (hoje: clínicas na Grande Vitória-ES e prestadores de serviço em Paraty-RJ, `vendas-config.json.faq`), por **R\$149/mês** (`vendas-config.json.pricing.base`), sem contrato de fidelidade, com domínio próprio como opcional pago à parte. O diferencial não é só "site bonito" — é que quem vende também tem a máquina de prospecção (Hunter) rodando por trás, o que uma agência ou freelancer comum não tem.

Como este documento se conecta aos outros

DOCUMENTO	O QUE COBRE
02-arquitetura.md	Como o sistema funciona de ponta a ponta, com diagramas
03-regras-de-negocio.md	Toda regra de negócio implementada, com arquivo/função
04-fluxos-e-modelo-comercial.md	O fluxo do lead até virar cliente, gargalos, diferenciais
05-kpis.md	Métricas que deveriam existir (hoje quase nada é medido)
06-roadmap-executivo.md	Prioridades em 30/90/180/360 dias
07-riscos.md	Riscos técnicos, comerciais, financeiros, operacionais
08-plano-melhoria-dados-e-experimentos.md	O que medir, como, e experimentos concretos
10-decisoes-estrategicas.md	Por que as decisões técnicas foram tomadas, e o que evitar

A apresentação de slides para o sócio está publicada separadamente (ver mensagem de entrega).

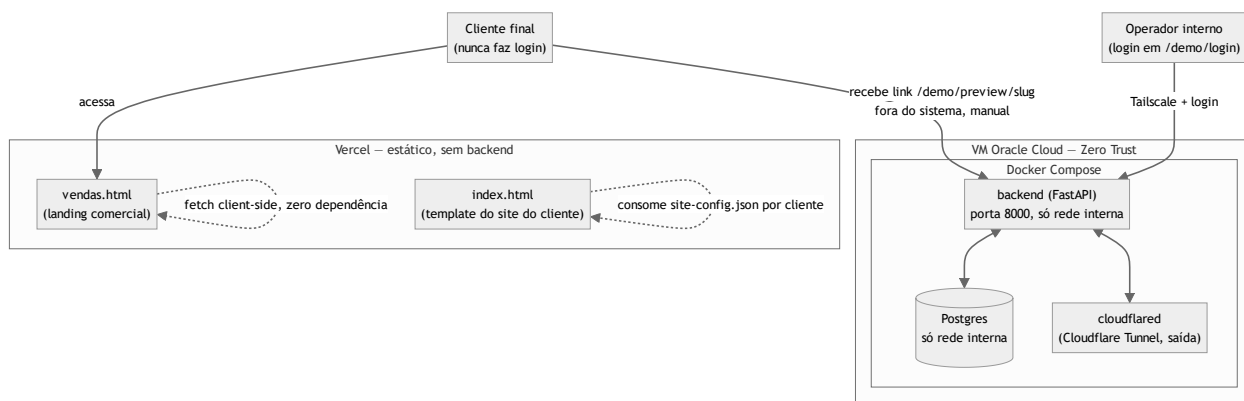
02 — Arquitetura

Fonte: leitura direta de `backend/app.py` , `backend/routers/*.py` , `Dockerfile` , `docker-compose.prod.yml` , `infra/zero_trust_deploy.md` , `index.html` , `vendas.html` . Diagramas descrevem o que o código faz hoje, não o que a documentação antiga (`CLAUDE.md`) descreve — ver seção "Divergência" ao final.

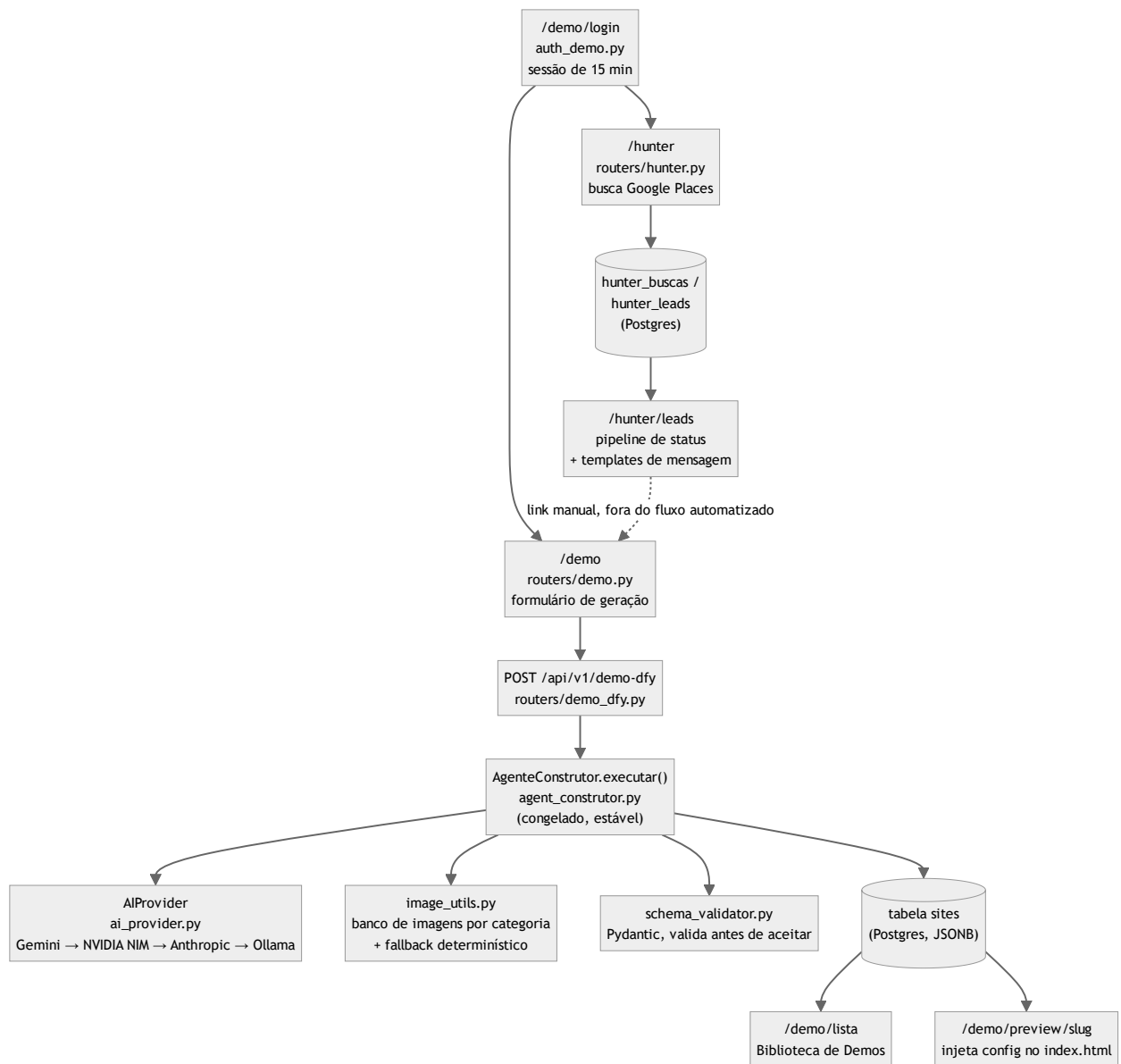
Visão geral: dois sistemas independentes

O projeto não é um site só — são **dois sites estáticos** (Vercel) e **um backend** (VM própria), com fronteiras deliberadas entre eles:

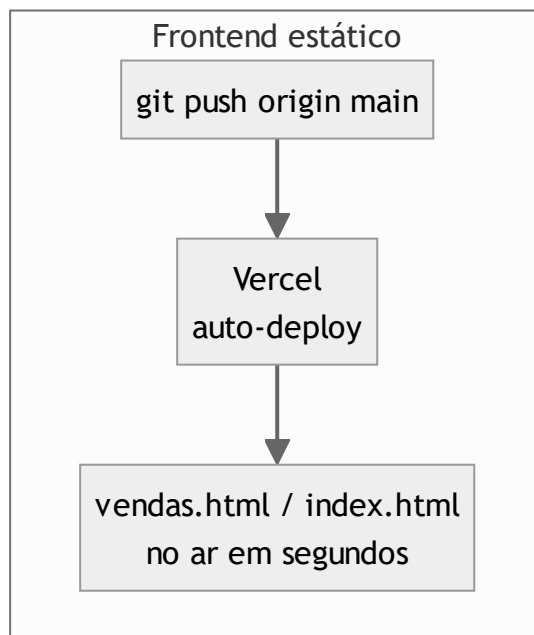
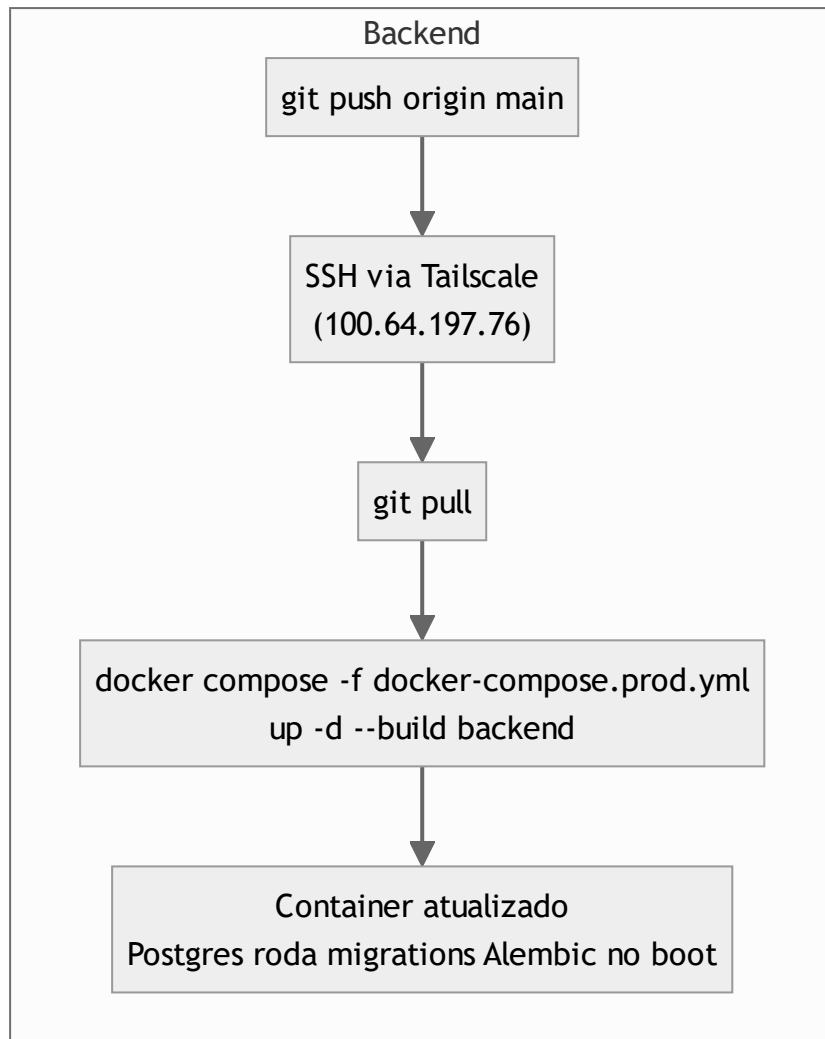
- **vendas.html** — a landing comercial (vende o produto). Zero dependência do backend: busca `vendas-config.json` direto do CDN da Vercel via `fetch()` . Continua no ar mesmo se o backend cair.
- **index.html** — o template que vira o site de cada cliente pago. Também estático, também na Vercel, injeta os dados de `site-config.json` (por cliente).
- **Backend FastAPI** (`backend/app.py`) — roda numa VM Oracle Cloud, nunca na Vercel. É onde vivem: o motor de geração de site, o Postgres, o Hunter, o login da ferramenta interna.



Fluxo interno do backend (ferramenta que o operador usa)

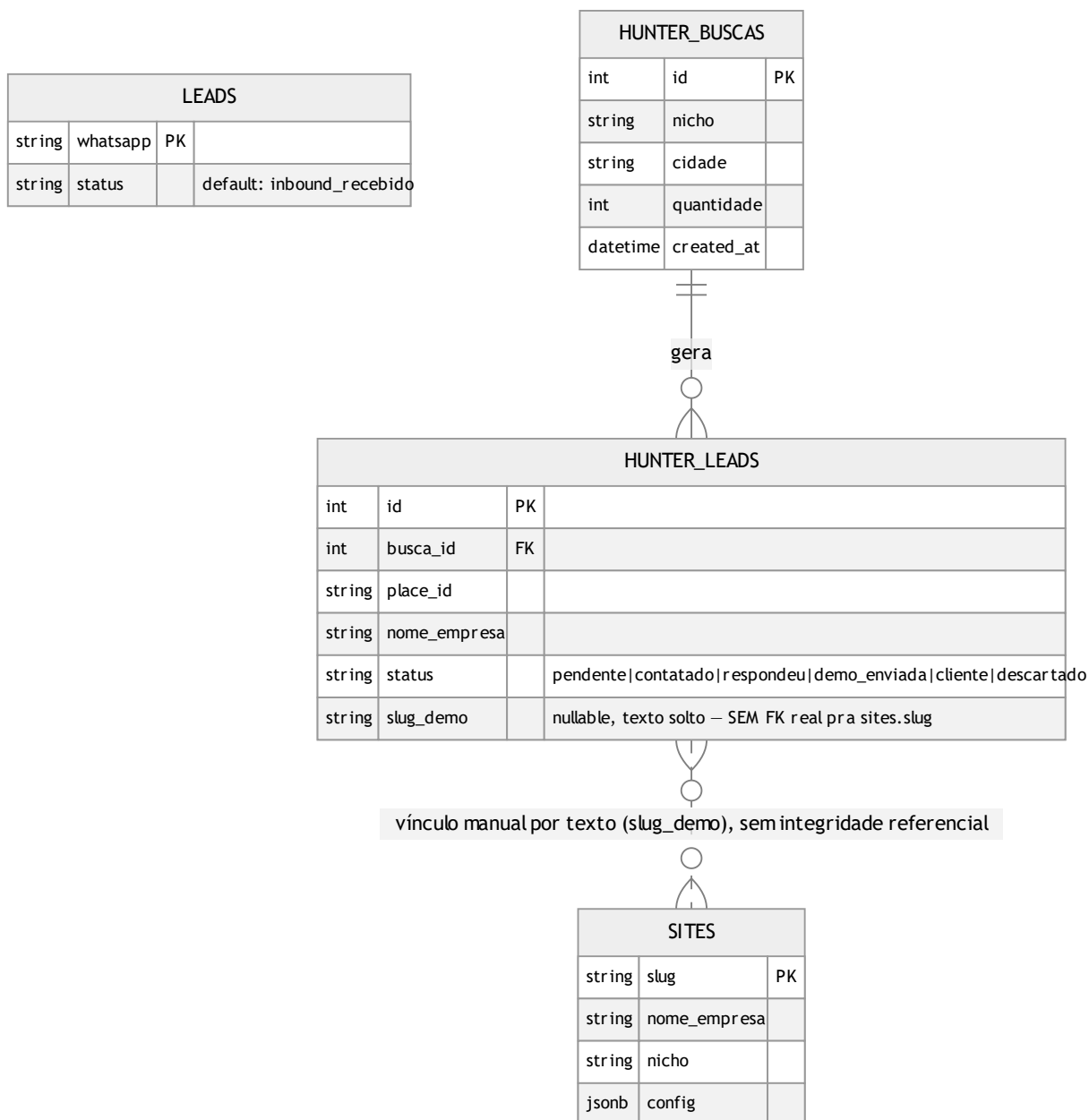


Deploy: dois pipelines independentes



Nenhum dos dois pipelines depende do outro. Um push que só mexe em `vendas.html` não precisa (e não deve) disparar rebuild do backend — e vice-versa.

Modelo de dados (Postgres, `backend/models_db.py`)



`LEADS` (webhook do WhatsApp) e `HUNTER_LEADS` (prospecção do Hunter) são tabelas **separadas e não relacionadas** hoje — um contato que chega pelo WhatsApp oficial não se cruza automaticamente com um lead que o Hunter encontrou.

Zero Trust — por que não há porta aberta

`docker-compose.prod.yml` não publica nenhuma porta de `db / backend` pro host (sem `ports:` nesses serviços — só existem na rede Docker interna `internal_net`). O único jeito de entrar é: - **Tailscale** (mesh privada) — pra administração da máquina (SSH). - **Cloudflare Tunnel** (`cloudflared` , saída apenas) — pra tráfego HTTP chegar no backend de fora, sem nunca abrir porta de entrada na VM.

Isso está documentado em `infra/zero_trust_deploy.md` e é a razão pela qual, por exemplo, tentar conectar direto num Postgres de produção a partir de uma máquina de desenvolvimento comum simplesmente não funciona — não é bug, é a arquitetura funcionando como projetada.

Divergência entre documentação antiga e arquitetura real

`CLAUDE.md` (raiz) e `ROADMAP.md` ainda descrevem o backend como pendente de deploy em **Render/Railway**. Isso foi substituído pela arquitetura Zero Trust em Oracle Cloud (commits a partir de `8b1e736` , documentado em `infra/zero_trust_deploy.md`), mas os dois documentos-fonte nunca foram atualizados para refletir essa mudança. Qualquer leitura futura desses dois arquivos (por uma pessoa ou por uma IA) deve desconfiar dessa seção específica até que seja corrigida.

03 — Regras de Negócio Implementadas

Toda regra abaixo foi confirmada lendo o código de fato (auditoria de 2026-07-24). Formato por regra: Objetivo, Onde está (arquivo/função), Como funciona, Quem usa, Impacto, Risco se falhar/for violada.

1. Preço único, fonte de verdade

- **Objetivo:** nunca ter dois lugares com preços diferentes pro mesmo produto.
- **Onde está:** `vendas-config.json.pricing.base` (o dado real) + `backend/routers/hunter.py::_preco_base() / _vendas_config()` (leitura, com `lru_cache`).
- **Como funciona:** o Hunter lê o preço do mesmo arquivo que a landing usa, em vez de ter um valor duplicado no Python. Se o arquivo não existir no container, a falha é alta e visível (`FileNotFoundError`), não silenciosa.
- **Quem usa:** `vendas.html` (client-side) e o template de oferta do Hunter (`TEMPLATE_OFERTA`).
- **Impacto:** alto — é a base de qualquer conversa comercial.
- **Risco se violada:** dois preços diferentes circulando (um na landing, outro numa mensagem de WhatsApp) — quebra de confiança na negociação.

2. Domínio próprio como opcional, não incluso

- **Objetivo:** não prometer no marketing algo que o plano base não cobre.
- **Onde está:** `vendas-config.json.pricing.opcionais` (item "Domínio próprio"), FAQ correspondente no mesmo arquivo.
- **Como funciona:** o plano base publica num endereço padrão; domínio `.com.br` é contratado à parte, sem preço fixo ainda decidido.
- **Quem usa:** landing (`vendas.html`).
- **Impacto:** médio — evita expectativa errada na venda.
- **Risco se violada:** cliente cobra "onde está meu domínio" depois de fechar, gerando atrito e retrabalho.

3. Garantia "sem letra miúda"

- **Objetivo:** reduzir a fricção de decidir comprar.
- **Onde está:** `vendas-config.json.pricing.garantia`.
- **Como funciona:** texto estático afirmando que dá pra começar só com o plano base, sem precisar contratar nenhum opcional.
- **Quem usa:** landing.
- **Impacto:** médio (conversão).

- **Risco se violada:** nenhum risco técnico — é só copy; o risco é comercial, se a operação não cumprir a promessa na prática.

4. Pipeline de status do lead (Hunter = CRM)

- **Objetivo:** rastrear onde cada lead prospectado está no funil comercial.
- **Onde está:** `backend/models_db.py::HunterLead.status`, valores geridos por `backend/repository.py`, UI em `/hunter/leads` (`backend/routers/hunter.py`).
- **Como funciona:** 6 estados — `pendente` → `contatado` → `respondeu` → `demo_enviada` → `cliente` → `descartado`. O avanço é **sempre manual**, via dropdown na UI que faz `POST /hunter/leads/{id}/status`. Nada no sistema avança um lead sozinho.
- **Quem usa:** o operador, na tela `/hunter/leads`.
- **Impacto:** alto — é literalmente o CRM da operação hoje.
- **Risco se violada/quebrada:** se o status não refletir a realidade (esquecimento de atualizar), o operador perde controle de quem já foi abordado e reaborda ou ignora leads por engano.

5. Trava de oferta por status

- **Objetivo:** impedir que a mensagem de oferta (mais "vendedora") seja enviada antes do primeiro contato ter sido respondido.
- **Onde está:** `backend/routers/hunter.py::STATUS_LIBERA_OFERTA = {"respondeu", "demo_enviada", "cliente"}`.
- **Como funciona:** o botão "Oferta" na tela `/hunter/leads` só fica clicável (não-desabilitado) quando o status do lead já é um dos três acima; antes disso, aparece cinza com tooltip explicando por quê.
- **Quem usa:** operador, na mesma tela do item 4.
- **Impacto:** médio — evita erro humano de sequência.
- **Risco se violada:** sem a trava, um operador apressado poderia mandar a oferta comercial completa pra alguém que nunca respondeu o primeiro "oi", o que soa invasivo e queima a abordagem.

6. Template de primeiro contato

- **Objetivo:** abordagem inicial padronizada, de baixa pressão, que não soa como spam comercial.
- **Onde está:** `backend/routers/hunter.py::TEMPLATE_ABORDAGEM / _mensagem()`.
- **Como funciona:** string com placeholders (`{nome}`, `{local}`, `{nichos_lower}`) — texto pronto pra copiar e colar manualmente no WhatsApp.
- **Quem usa:** botão "1º contato" em `/hunter` e `/hunter/leads`.
- **Impacto:** médio — padroniza a qualidade da primeira mensagem, independente de quem está operando.

- **Risco se violada:** inconsistência de tom entre diferentes pessoas abordando leads.

7. Template de oferta pós-resposta

- **Objetivo:** converter interesse demonstrado numa proposta comercial concreta.
- **Onde está:** `backend/routers/hunter.py::TEMPLATE_OFERTA / _mensagem_oferta()`.
- **Como funciona:** usa o preço dinâmico (regra 1) e os benefícios reais do plano base — não é gerado por IA, é template estático.
- **Quem usa:** botão "Oferta", condicionado pela regra 5.
- **Impacto:** médio-alto — é o momento em que a conversa vira proposta.
- **Risco se violada:** ver regra 5 (a trava é o que evita o uso indevido).

8. Nunca disparo automático de WhatsApp

- **Objetivo:** evitar banimento do número pela Meta por envio em massa sem opt-in explícito — risco citado como guardrail central desde o `ROADMAP.md`.
- **Onde está:** confirmado ausente em todo o código revisado — `hunter.py` (templates só geram texto pra copiar), `backend/agents/vendedor.py` (mock de envio, nunca chama API real), `backend/routers/whatsapp_inbound.py` (só recebe e grava, nunca responde sozinho).
- **Como funciona:** toda "mensagem sugerida" em qualquer parte do sistema é texto estático que um humano decide se envia, quando envia, e envia manualmente.
- **Quem usa:** todo o sistema — é uma regra transversal, não de um módulo específico.
- **Impacto:** crítico.
- **Risco se violada:** perda do número de WhatsApp comercial (banimento), que hoje é o canal de vendas inteiro da operação.

9. Nunca automatizar venda antes do gate de 15-20 clientes fechados

- **Objetivo:** não treinar/confiar um agente automatizado de vendas com um roteiro nunca validado com cliente real.
- **Onde está:** guardrail do `ROADMAP.md`, respeitado em `backend/agents/vendedor.py` (toda função de envio é mock).
- **Como funciona:** o agente Vendedor tem lógica real de scoring/mensagem, mas nenhuma chamada HTTP de verdade sai dele hoje.
- **Quem usa:** `AgenteVendedor` (ainda não plugado em produção real).
- **Impacto:** alto — é uma proteção deliberada contra escalar um processo não testado.
- **Risco se violada:** vender mal, em escala, com um script que nunca foi validado por um humano numa negociação real.

10. Nunca fabricar depoimento, contato ou rede social no site gerado

- **Objetivo:** garantir que todo site gerado pela IA seja honesto — nenhum dado inventado aparece pro cliente final do cliente.
- **Onde está:** `backend/agent_construtor.py::_preencher_fallbacks()` (zera `email / social / testimonials` sempre, independente do que o modelo de IA tenha retornado), documentado também em `backend/schema_validator.py`.
- **Como funciona:** é um guardrail determinístico, não uma instrução de prompt que a IA pode ou não seguir — o código força esses campos vazios depois da geração.
- **Quem usa:** todo site gerado via `AgenteConstrutor`.
- **Impacto:** alto — é o que garante que o produto nunca entregue uma mentira pro cliente final (ex.: um depoimento que ninguém deu).
- **Risco se violada:** cliente descoberto com informação falsa no próprio site, prejuízo de reputação direto pro negócio dele e indireto pra Fábrica de Sites.

11. Sessão de login interno curta (15 minutos)

- **Objetivo:** limitar a janela de exposição da ferramenta interna (Hunter, Site Constructor, Biblioteca de Demos).
- **Onde está:** `backend/auth_demo.py::SESSAO_DURACAO_SEGUNDOS = 900`; comparação de credencial via `hmac.compare_digest` (defesa contra timing attack).
- **Como funciona:** qualquer requisição pras rotas internas (`/demo`, `/demo/lista`, `/hunter`, `/demo/preview`) checa a sessão e redireciona pro login se expirou.
- **Quem usa:** todas as rotas internas.
- **Impacto:** médio.
- **Risco se violada/mal calibrada:** 15 minutos pode ser curto pra um fluxo de geração de demo mais longo (o próprio formulário já documentou timeout de geração de 30-60s) — risco é de fricção operacional, não de segurança.

12. Conciliação de pagamento estrita (PIX)

- **Objetivo:** só liberar acesso ao site quando o pagamento estiver **de fato** confirmado, nunca por suposição.
- **Onde está:** `backend/agents/financeiro.py::conciliar()`.
- **Como funciona:** exige status "aprovado" e valor batendo (tolerância de 1 centavo); qualquer valor a maior cai em revisão manual em vez de aprovar automaticamente.
- **Quem usa:** `AgenteFinanceiro` — **ainda sem webhook de gateway real plugado em produção**, é lógica pronta esperando integração.
- **Impacto:** alto, quando ativado.
- **Risco se violada:** liberar um site sem pagamento real de fato ter entrado.

13. Deduplicação de leads do Hunter

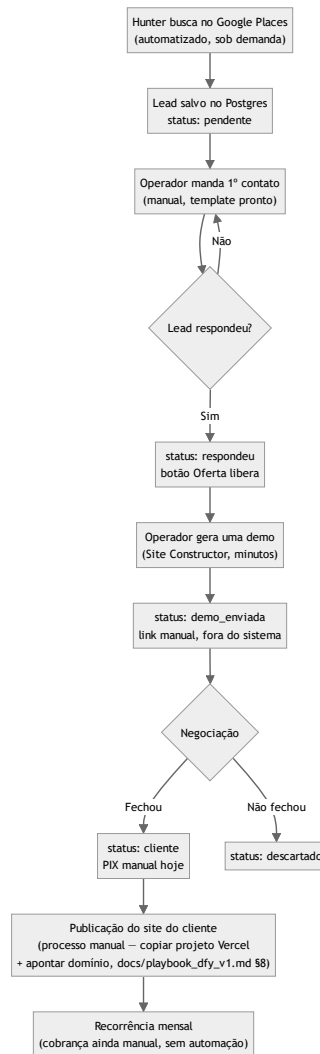
- **Objetivo:** não abordar a mesma empresa duas vezes como se fosse um lead novo.
- **Onde está:** `backend/repository.py::_lead_hunter_existente()` .
- **Como funciona:** checa por `place_id` (preferencial, vem do Google Places) ou, na ausência dele, por `(nome_empresa, cidade)` .
- **Quem usa:** `salvar_leads_hunter()` , chamado toda vez que uma busca nova é feita.
- **Impacto:** médio — evita parecer desorganizado com um lead que já foi contatado.
- **Risco se violada:** reabordagem duplicada, ou pior, dois operadores diferentes abordando o mesmo lead com mensagens diferentes.

Regras que existem só como especificação, ainda sem código (não confundir com as acima): cobrança recorrente automática (`docs/fluxo_financeiro_recorrencia.md`), pipeline inteligente de seleção de imagem (`docs/pipeline_imagens_inteligente.md`), evolução comercial do Hunter com scoring avançado (`docs/hunter_online_spec.md` — mas atenção: a spec descreve como "futuro" um pipeline de status que **já existe e está em produção**, ver regra 4 acima; a spec não foi atualizada).

04 — Fluxo do Negócio e Modelo Comercial

Fluxo real, como a empresa funciona hoje

Fonte: código (`backend/routers/hunter.py` , `demo.py` , `demo_dfy.py` , `demo_preview.py`) + processo manual documentado em `docs/playbook_dfy_v1.md` . Onde uma etapa é manual/fora do sistema, está marcado.



Onde estão os gargalos, hoje (interpretação com base no que o código automatiza vs. não automatiza):

ETAPA	AUTOMATIZADO?	GARGALO REAL
Encontrar leads	✅ Sim (Hunter)	Nenhum — o sistema já encontra mais leads do que a operação consegue trabalhar
Abordar (1º contato)	🟡 Template pronto, envio manual	Depende de tempo humano disponível
Gerar demo	✅ Sim (minutos)	Nenhum
Fechar venda	🔴 100% manual/humano	Este é o gargalo real — não há dado no repositório mostrando quantas vendas foram fechadas até hoje
Publicar site do cliente	🔴 Processo manual (copiar projeto Vercel, apontar domínio)	Escala mal se o volume de clientes crescer rápido — hoje é aceitável no volume atual
Cobrar recorrência	🔴 Não existe (nem manual documentado como rotina)	Risco de churn silencioso por falta de cobrança consistente

Riscos do fluxo atual: - Publicação manual de site por cliente não escala além de dezenas de clientes sem virar gargalo operacional. - Sem cobrança recorrente automatizada, a receita mensal depende de alguém lembrar de cobrar — risco real de vazamento de faturamento. - Sem uma tabela relacionando `hunter_leads` a `sites` por chave estrangeira real (só um campo de texto solto, `slug_demo`), rastrear "qual lead virou qual cliente" depende de disciplina manual, não de garantia do banco.

Oportunidades visíveis no código, ainda não usadas: - O agente `AgenteFinanceiro` (`backend/agents/financeiro.py`) já tem lógica de conciliação PIX pronta — falta só plugar um webhook de gateway real (Asaas é a recomendação já registrada em `docs/fluxo_financeiro_recorrencia.md`). - O agente `AgenteVendedor` já tem lógica de mensagem/scoring pronta — quando o gate de 15-20 vendas manuais for atingido, a automação de follow-up pode ser ligada sem reescrever nada, só trocando o mock por uma chamada real.

Modelo Comercial

Esta seção é síntese estratégica — a origem do dado é `vendas-config.json` (posicionamento e preço reais publicados) combinada com decisões de negócio já registradas no histórico do projeto. Está marcada como interpretação, não como fato de código.

O que vendemos: um site profissional, gerado por IA, para pequenos negócios locais que ainda não têm presença digital — não é "mais um construtor de sites", é a combinação de descobrir quem precisa (Hunter) + entregar rápido (Agente Construtor).

Quem compra (hoje): negócios locais nos nichos e regiões já validados — clínicas na Grande Vitória-ES e prestadores de serviço em Paraty-RJ (`vendas-config.json.faq` , resposta sobre atendimento fora de clínicas/consultórios).

Cliente ideal: um pequeno negócio que já tem clientes por indicação/Instagram, mas nenhuma presença que apareça no Google — ou seja, alguém que perde cliente novo especificamente por não ser encontrado, não por falta de qualidade no serviço que presta.

Proposta de valor: site no ar rápido, preço de entrada baixo (R\$149/mês), sem contrato de fidelidade, com WhatsApp como canal central de conversão (todo botão de ação no site leva pro WhatsApp, `vendas-config.json.differentials`).

Por que somos diferentes de:

ALTERNATIVA	DIFERENÇA REAL (COM BASE NO QUE ESTÁ IMPLEMENTADO)
Freelancer	Processo repetível via <code>agent_construtor.py</code> — qualidade não depende de disponibilidade de uma pessoa específica
Agência tradicional	Site em minutos, não semanas — o motor de geração é o diferencial de velocidade

ALTERNATIVA	DIFERENÇA REAL (COM BASE NO QUE ESTÁ IMPLEMENTADO)
Wix/WordPress	O cliente não edita nada — recebe pronto, sem precisar aprender ferramenta nenhuma
Hostinger/construtores genéricos	Prospecção embutida (Hunter) — não é só "construa seu site", é "nós achamos quem precisa e já entregamos pronto"
IA genérica (ChatGPT + template)	Todo o pipeline (schema validado, fallback de imagem garantido, anti-fabricação de depoimento) é construído especificamente pra nunca entregar um site quebrado ou com dado inventado — ver regras 10 em 03-regras-de-negocio.md

Ressalva honesta: nenhuma dessas comparações tem prova de mercado (pesquisa de concorrente, dado de conversão comparado) no repositório — é a lógica de produto, não um dado medido.

05 — KPIs

Importante: nenhum dashboard de métricas de negócio existe hoje no código (confirmado na auditoria — Fase 4 do ROADMAP.md , "não iniciada"). O que existe é backend/metrics.py , que grava eventos técnicos em arquivo local (log + JSON), sem agregação nem visualização. Os KPIs abaixo são um **framework proposto**, não uma métrica já calculada em algum lugar — cada um indica de onde o dado viria, com base no que já está no banco hoje.

Aquisição

KPI	COMO CALCULAR	DE ONDE VEM O DADO HOJE
Leads encontrados	COUNT(*) em hunter_leads	Já existe na tabela (backend/models_db.py :HunterLead)
Leads por busca	hunter_buscas.quantidade vs. leads salvos de fato	Já existe (hunter_buscas)
Leads por nicho/cidade	GROUP BY nicho, cidade em hunter_leads	Já existe

Conversão

KPI	COMO CALCULAR	DE ONDE VEM O DADO HOJE
Taxa de resposta	status != 'pendente' ÷ total	Já existe (campo status)
Taxa de demonstração	status IN ('demo_enviada','cliente') ÷ status = 'respondeu'	Já existe
Taxa de venda	status = 'cliente' ÷ total contatado	Já existe
Tempo médio pendente→cliente	updated_at só grava a última mudança, não um histórico — precisaria de uma tabela de histórico de status pra medir isso direito	Não existe hoje — hunter_leads só guarda o status atual, não transições anteriores

Operação

KPI	COMO CALCULAR	DE ONDE VEM O DADO HOJE
Demos geradas por dia/semana	<code>COUNT(*)</code> em <code>sites</code> por <code>created_at</code> (△ campo a confirmar no schema real de <code>sites</code>)	Parcial — a tabela existe, mas não há confirmação de campo de data de criação lido na auditoria
Tempo de geração de demo	Não instrumentado	Não existe — precisaria logging de início/fim ao redor de <code>AgenteConstrutor.executar()</code>

Financeiro

KPI	COMO CALCULAR	DE ONDE VEM O DADO HOJE
MRR (receita recorrente mensal)	<code>clientes_ativos</code> × R\$149 (hoje sem variação de plano)	Não existe tabela de assinatura — <code>docs/fluxo_financeiro_recorrencia.md</code> propõe isso, zero código ainda
Churn	Clientes que saem ÷ total	Não existe — depende da tabela de assinatura acima existir primeiro
CAC (custo de aquisição)	Custo de operação (tempo humano + Google Places API) ÷ clientes fechados	Não existe — nenhum dado de custo por lead é registrado hoje
Ticket médio / LTV	Depende de MRR + churn existirem	Não existe

Produto

KPI	COMO CALCULAR	DE ONDE VEM O DADO HOJE
Sites gerados com sucesso na 1ª tentativa	<code>MAX_TENTATIVAS_GERACAO</code> (<code>agent_construtor.py</code>) já registra retry — dá pra contar quantas gerações precisaram de mais de 1 tentativa	Parcial — o mecanismo de retry existe, mas não há um contador agregado persistido
Taxa de erro de geração	Idem acima	Parcial

Satisfação

KPI	COMO CALCULAR	DE ONDE VEM O DADO HOJE
Qualquer medida de satisfação do cliente final	—	Não existe nenhum mecanismo de coleta — nenhuma pesquisa, nenhum campo de feedback em nenhuma tabela

Leitura honesta desta seção

A maior parte do funil **comercial** (aquisição → conversão) já tem o dado bruto disponível no Postgres hoje — só falta uma consulta e uma visualização, não uma mudança de arquitetura. A maior lacuna real está no lado **financeiro** (nenhuma tabela de assinatura/pagamento existe) e no **histórico de transições de status** (só o estado atual é guardado, perdendo a linha do tempo de cada lead). Ver [08-plano-melhoria-dados-e-experimentos.md](#) pra como fechar essas lacunas sem over-engineering.

06 — Roadmap Executivo

Prioridades organizadas por horizonte de tempo. Baseado no estado real observado na auditoria (não no ROADMAP.md como está escrito hoje, que tem itens desatualizados — ver [10-decisoes-estrategicas.md](#)). O critério de priorização é o mesmo já adotado no projeto: qualquer item passa pelo filtro *ajuda a vender?* / *ajuda a operação diária?* / *é só estética?* antes de entrar aqui.

30 dias

Comercial - Focar 100% do tempo disponível em trabalhar o pipeline do Hunter que já existe — encontrar leads não é o gargalo, converter é. - Registrar manualmente (mesmo que numa planilha simples) quantas vendas foram fechadas — hoje não há esse número em lugar nenhum do sistema, e é a métrica mais importante do negócio.

Produto - Nenhuma mudança visual nova na landing — gate já registrado (só evoluir com feedback real de lead).

Tecnologia - Corrigir `.env local ( DATABASE_URL` aponta pra uma VM Oracle já desligada). - Atualizar `CLAUDE.md / ROADMAP.md` pra refletir a infraestrutura Zero Trust real. - Restringir CORS (`allow_origins=["*"]`) e adicionar rate limit básico.

Operação - Nenhuma mudança de processo — o gargalo é volume de conversas humanas, não ferramenta.

Marketing - Nenhum item novo — a landing v1 está no ar e o próprio processo já define que só muda de novo com sinal real de lead.

90 dias

Comercial - Atingir o gate de 15-20 vendas fechadas manualmente (marco explícito do ROADMAP.md pra liberar qualquer automação de venda).

Produto - Se o volume de clientes crescer, visitar o processo manual de publicação (copiar projeto Vercel + apontar domínio) — hoje aceitável, não escala bem além de algumas dezenas de clientes.

Tecnologia - Criar uma tabela de histórico de transições de status do Hunter (hoje só o estado atual é salvo) — sem isso, não dá pra medir tempo médio de conversão. - Avaliar a primeira versão de uma tabela de assinatura (`customers / subscriptions`), mesmo que a cobrança em si continue manual no início.

Operação - Se o volume justificar, considerar automatizar o **follow-up** (não a venda em si) do AgenteVendedor — ele já tem lógica pronta, só falta plugar.

Marketing - Nenhum novo canal — validar o que já existe antes de multiplicar canal.

6 meses

Comercial - Se o gate de 15-20 vendas foi atingido, avaliar ligar o `AgenteVendedor` de fato (troca de mock por chamada real), com acompanhamento humano próximo no início.

Produto - Avaliar se a Biblioteca de Demos precisa do campo "status de negociação" (item já identificado numa auditoria anterior do app interno como o único achado de alto valor, ver `docs/histórico do projeto`).

Tecnologia - Implementar cobrança recorrente real (Asaas é a recomendação já registrada em `docs/fluxo_financeiro_recorrencia.md`) — só depois de haver clientes suficientes pra justificar. - Backup automatizado do Postgres de produção — hoje não existe nenhum mecanismo identificado.

Operação - Se o volume de clientes exigir, revisitar o processo de publicação manual do site do cliente (ver 90 dias).

Marketing - Reavaliar a landing com base em feedback real acumulado (não antes disso).

12 meses

Comercial - Expandir nicho/região só depois de o modelo atual estar validado com receita recorrente real e sustentada.

Produto - Avaliar Fase 4 (dashboard de negócio) — só faz sentido quando já existir volume de dado real o suficiente pra um dashboard ter algo a mostrar.

Tecnologia - Reavaliar se algum dos três agentes especializados precisa de uma camada de configuração compartilhada — usando a regra já estabelecida no projeto (1 consumidor mantém local, 2 considera extrair, 3+ extrai).

Operação - Se o volume de clientes justificar, considerar automação real da conciliação financeira (`AgenteFinanceiro` já tem a lógica pronta).

Marketing - Considerar expansão de canal só com dado de CAC/LTV real em mãos — nenhum dos dois existe hoje (ver [05-kpis.md](#)).

Princípio que atravessa todos os horizontes: nenhum item aqui propõe nova arquitetura antes da hora. Cada item de 90+/6/12 meses só avança se o horizonte anterior comprovar que o volume real do negócio justifica — a mesma disciplina que já evitou construir um "Business Rules Engine" prematuro nesta mesma semana (ver [10-decisoes-estrategicas.md](#)).

07 — Riscos

Baseado na auditoria de 2026-07-24. Probabilidade e impacto são estimativas qualitativas (Baixo/Médio/Alto), não um cálculo formal.

Riscos técnicos

RISCO	PROBABILIDADE	IMPACTO	MITIGAÇÃO
CORS aberto (<code>allow_origins= ["*"]</code> , <code>backend/app.py:89</code>) explorado por outro domínio	Baixa hoje (baixo tráfego)	Médio	Restringir a origens conhecidas antes de crescer volume
Ausência de rate limit em <code>/api/v1/generate-site</code> e nos webhooks	Baixa hoje	Médio-Alto (custo de API de IA em caso de abuso)	Adicionar rate limit básico por IP/API key
Sem backup documentado do Postgres de produção	Média (qualquer falha de disco/operação humana)	Alto — perda de todos os leads e sites gerados	Rotina de backup automatizado, mesmo que simples (dump diário)
Container do backend roda como root (<code>Dockerfile</code> sem <code>USER</code> não-root)	Baixa	Médio (se houver RCE, o blast radius é maior)	Adicionar usuário não-root na imagem
<code>hunter_leads.slug_demo</code> sem FK real pra <code>sites.slug</code>	Já acontece hoje (dado solto)	Baixo-Médio (perda de rastreabilidade, não de dado)	Adicionar FK real quando o volume justificar o refactor
Sessão de login interno de 15 min pode ser curta pro fluxo de geração	Média (fricção operacional)	Baixo	Ajustar duração se virar reclamação recorrente do operador

Riscos comerciais

RISCO	PROBABILIDADE	IMPACTO	MITIGAÇÃO
Banimento do número de WhatsApp por envio indevido	Baixa enquanto o guardrail de nunca automatizar envio for respeitado	Crítico — perde o canal de vendas inteiro	Manter a disciplina de mensagem manual até ter BSP oficial + opt-in real

RISCO	PROBABILIDADE	IMPACTO	MITIGAÇÃO
Automatizar vendas antes do gate de 15-20 clientes	Só acontece se alguém decidir pular o guardrail	Alto (vender mal, em escala, com script não validado)	Manter o gate como decisão consciente, não just "porque dá pra automatizar"
Nenhum dado de quantas vendas foram fechadas até hoje	Já é realidade	Médio (decisão estratégica sem visibilidade real do funil)	Registrar manualmente até existir instrumentação
Processo de publicação manual do site do cliente não escala	Cresce com o volume de clientes	Médio	Revisitar quando o volume de clientes justificar (ver roadmap 90 dias)

Riscos financeiros

RISCO	PROBABILIDADE	IMPACTO	MITIGAÇÃO
Cobrança recorrente 100% manual	Já é realidade	Alto — vazamento de receita por esquecimento	Priorizar cobrança automatizada assim que houver volume que justifique (Asaas já recomendado)
Sem CAC/LTV calculado	Já é realidade	Médio — decisão de investimento em aquisição sem base de dado	Instrumentar o funil (ver KPIs) antes de qualquer decisão de investir mais em prospecção

Riscos operacionais

RISCO	PROBABILIDADE	IMPACTO	MITIGAÇÃO
Documentação central (CLAUDE.md / ROADMAP.md) desatualizada sobre a infraestrutura real	Já é realidade	Médio — decisão futura (humana ou de IA) baseada em informação errada	Atualizar os dois documentos pra refletir Oracle Zero Trust
8 documentos obsoletos na raiz do repo, de uma versão anterior do produto	Já é realidade	Baixo-Médio — risco de alguém seguir instrução desatualizada	Arquivar/remover, não fica ambíguo pra próxima pessoa que ler

RISCO	PROBABILIDADE	IMPACTO	MITIGAÇÃO
Nenhum monitoramento/alerta em produção (só log local em arquivo dentro do container)	Já é realidade	Médio — um problema em produção só é percebido se alguém for olhar manualmente	Considerar alerta simples (mesmo que só um healthcheck externo) quando o negócio depender mais da disponibilidade
Dependência de uma única pessoa saber operar o deploy (SSH manual via Tailscale)	Já é realidade	Médio (bus factor)	Documentar o processo de deploy de forma que outra pessoa consiga repetir (<code>infra/zero_trust_deploy.md</code> já existe — validar se está atualizado e completo)

08 — Dados, Melhoria Orientada por Dados e Experimentos

O que o sistema coleta hoje

Fonte: `backend/models_db.py` (schema real das 4 tabelas).

- **sites** : todo site gerado (slug, nome da empresa, nicho, config completo em JSON).
- **leads** : contato que chegou pelo webhook oficial do WhatsApp (`whatsapp` , `status`) — hoje só grava, nada mais.
- **hunter_buscas** : toda execução de busca do Hunter, mesmo sem resultado (nicho, cidade, quantidade pedida, data).
- **hunter_leads** : cada empresa encontrada, com status atual do pipeline comercial e link solto (texto) pra um site gerado, se houver.

O que ainda NÃO coleta

- **Histórico de transição de status** — `hunter_leads.status` só guarda o estado atual; quando um lead passa de "pendente" pra "contatado", a informação de "quando" e "quanto tempo ficou em cada estado" se perde.
- **Motivo de descarte** — quando um lead vira `status = 'descartado'` , não há campo pra registrar por quê (preço, não respondeu mais, já tinha site, etc.).
- **Qualquer dado financeiro** — não existe tabela de assinatura, pagamento ou cobrança.
- **Qualquer dado de satisfação do cliente final** — nenhum mecanismo de coleta de feedback.
- **Tempo de geração de demo** — o motor de geração não registra quanto tempo levou nem quantas tentativas de retry foram necessárias, de forma persistida e consultável.
- **Motivo de abandono após receber a demo** — se um cliente vê a demo e não segue adiante, o sistema não pergunta por quê nem registra.

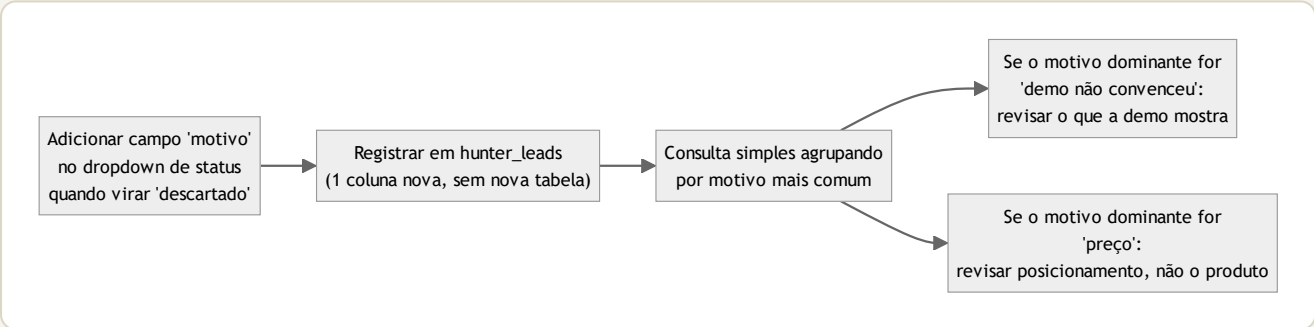
Classificação (Crítico / Importante / Futuro)

DADO A COLETAR	CLASSIFICAÇÃO	POR QUÊ
Motivo de descarte de lead	Crítico	É a informação mais barata de coletar (um campo a mais no formulário de mudança de status) e a mais valiosa pra melhorar a abordagem
Histórico de transição de status	Crítico	Sem isso, nenhum KPI de tempo de conversão é possível

DADO A COLETAR	CLASSIFICAÇÃO	POR QUÊ
Vendas fechadas (número real)	Crítico	Não existe hoje em lugar nenhum — é a métrica mais básica do negócio
Motivo de abandono pós-demo	Importante	Ajuda a melhorar a geração/apresentação da demo
Dado financeiro (assinatura/pagamento)	Importante	Necessário antes de qualquer automação de cobrança
Tempo de geração / taxa de retry	Futuro	Só importa quando o volume de geração for alto o suficiente pra virar gargalo perceptível
Satisfação do cliente final	Futuro	Só faz sentido depois de haver uma base de clientes ativa o bastante pra pesquisar

Como transformar dado em melhoria (exemplo do próprio pedido do sócio)

"Muitos clientes abandonam após receber a demo" — fluxo de como isso viraria melhoria real, sem introduzir IA onde não precisa:



Nenhuma etapa desse fluxo pede IA nova — é campo de formulário + consulta SQL + decisão humana.

Experimentos propostos

Cada experimento seguindo o mesmo formato: hipótese, como medir, métrica, critério de sucesso, tempo, rollback.

Experimento 1 — Motivo de descarte

- **Hipótese:** a maioria dos leads descartados hoje é por um motivo concentrado (ex.: "já tem site" ou "não respondeu"), o que mudaria a forma de qualificar antes de abordar.
- **Como medir:** adicionar campo obrigatório de motivo ao marcar um lead como `descartado`.

- **Métrica:** distribuição percentual dos motivos após 30 dias de uso.
- **Critério de sucesso:** um motivo concentra mais de 40% dos descartes, indicando uma ação clara (ex.: filtrar melhor na busca do Hunter).
- **Tempo:** 30 dias de coleta.
- **Rollback:** campo opcional, não bloqueia o fluxo se ninguém preencher — reversível apenas removendo a coluna, sem impacto em dado existente.

Experimento 2 — Tempo de resposta ao 1º contato

- **Hipótese:** leads abordados dentro de X horas do momento em que foram encontrados respondem mais do que os abordados depois.
- **Como medir:** usar `hunter_leads.created_at` (quando o lead foi salvo) vs. o momento em que o status muda pra `contatado` — precisa do histórico de transição do item "Crítico" acima primeiro.
- **Métrica:** taxa de resposta por faixa de tempo até o 1º contato.
- **Critério de sucesso:** diferença de taxa de resposta clara entre "abordado no mesmo dia" vs. "abordado depois de 3+ dias".
- **Tempo:** 60 dias (precisa de volume mínimo de leads contatados).
- **Rollback:** é só análise, nenhuma mudança de sistema, nada a reverter.

Experimento 3 — Efeito da mensagem de oferta

- **Hipótese:** leads que recebem a mensagem de oferta (`TEMPLATE_OFERTA`) logo após responder convertem mais do que os que demoram a receber.
- **Como medir:** cronometrar manualmente por enquanto (não há histórico de transição ainda) o intervalo entre `respondeu` e `demo_enviada / cliente`.
- **Métrica:** taxa de conversão por faixa de tempo de resposta da oferta.
- **Critério de sucesso:** sinal claro de que agilidade importa (ou não) — qualquer resultado é útil aqui, mesmo um "não importa muito".
- **Tempo:** 30-60 dias.
- **Rollback:** nenhum — é só observação de processo já existente, não uma mudança de sistema.

Regra que vale pra qualquer experimento futuro: nunca propor IA generativa nova pra resolver um problema que uma consulta SQL, um campo de formulário ou uma decisão humana já resolvem — essa é a mesma disciplina que já rejeitou um "Business Rules Engine" prematuro nesta mesma semana de trabalho (ver [10-decisoes-estrategicas.md](#)).

10 — Decisões Estratégicas do Projeto

Por que cada decisão de arquitetura foi tomada

DECISÃO	POR QUÊ	ONDE ESTÁ
HTML estático + Tailwind via CDN, sem build	O site institucional (vendas.html) e o template de cliente (index.html) nunca precisam de build/bundler pra funcionar — cada mudança de conteúdo é só editar um JSON. Menos infraestrutura, menos coisa pra quebrar, deploy é literalmente git push	index.html , vendas.html , ausência de vercel.json / package.json de app
Sem React/Next.js	O produto vendido (site do cliente) e a landing não têm necessidade de interatividade complexa que justifique um framework — foi uma proposta externa avaliada e rejeitada explicitamente durante o redesign da landing (2026-07-22)	Histórico de decisão registrado em memória de projeto
Sem dashboard interno grande	Login vai direto pro Site Constructor — decisão de manter simples até haver volume de dado que realmente precise de um painel	backend/routers/demo.py , ausência de rota /dashboard
Zero Trust (Tailscale + Cloudflare Tunnel + Oracle, NSG fechado)	Nenhuma porta de entrada exposta na VM — o único jeito de acessar é por uma rede privada (administração) ou por um túnel de saída (tráfego HTTP). Elimina a superfície de ataque de portas abertas na internet	docker-compose.prod.yml , infra/zero_trust_deploy.md
Cloudflare Tunnel em vez de porta pública	Permite servir HTTP sem NUNCA abrir uma porta de entrada na VM — o tráfego sai de dentro da VM em direção ao Cloudflare, não o contrário	infra/zero_trust_deploy.md

DECISÃO	POR QUÊ	ONDE ESTÁ
Oracle Cloud (não Render/Railway como planejado originalmente)	Substituição de infraestrutura já feita na prática (<code>docker-compose.prod.yml</code> real, VM provisionada) — decisão tomada mas nunca propagada de volta pros documentos (<code>CLAUDE.md</code> / <code>ROADMAP.md</code> ainda citam a opção antiga)	<code>infra/</code>
Vercel pro frontend estático	Deploy automático a cada <code>git push</code> , zero servidor pra manter pro que não precisa de servidor	Configuração do projeto na Vercel (fora do repo)
DFY ("Done For You" — a equipe gera e entrega, o cliente não mexe em nada)	O cliente-alvo (pequeno negócio local) não quer aprender ferramenta nenhuma — quer o site pronto e funcionando	<code>docs/playbook_dfy_v1.md</code>
Sem over-engineering / abstração prematura	Princípio explícito do <code>CLAUDE.md</code> : "para tarefas simples, prefira soluções diretas" — reforçado nesta mesma semana ao rejeitar um "Business Rules Engine" completo em favor de uma correção de poucas linhas	<code>CLAUDE.md</code> , ver caso concreto abaixo
Fonte única de preço (<code>vendas-config.json</code>)	Eliminar duplicação de dado sem criar uma camada de configuração nova — o backend passou a ler o mesmo arquivo que a landing usa, em vez de ter o preço hardcoded duas vezes	<code>backend/routers/hunter.py::_preco_base()</code> , <code>Dockerfile</code>
Congelamento do Agente Construtor	O motor de geração atingiu 100% de taxa de sucesso em teste (<code>CLAUDE.md</code>) — mexer sem necessidade explícita é risco puro, sem ganho	<code>CLAUDE.md</code> , <code>backend/agent_construtor.py</code>

Caso concreto desta semana: o Business Rules Engine que não foi construído

Um exemplo real e recente de disciplina de escopo, útil pra ilustrar como decisões como as acima são tomadas na prática: ao corrigir uma duplicação de preço entre a landing e o Hunter, uma proposta externa sugeriu construir um "Business Rules Engine" completo — 8 arquivos novos, YAML

de regras, motor de workflow, scoring, mandato de 90% de cobertura de teste. A avaliação, usando o mesmo critério de sempre (ajuda a vender? ajuda a operação hoje? é proporcional ao estágio da empresa?), rejeitou a proposta: o projeto tem **um plano de preço real e zero clientes pagantes documentados** — construir uma "engine de regras" pra esse estágio seria meses de trabalho de infraestrutura sem nenhum cliente novo em troca. A correção real aplicada foi de duas linhas de lógica: o Hunter passou a ler o preço do mesmo arquivo que a landing já usa (vendas-config.json), sem criar nenhum arquivo de configuração novo.

O que NÃO devemos fazer (armadilhas já identificadas no histórico do projeto)

ARMADILHA	POR QUE É UMA ARMADILHA AQUI
Criar módulos "porque um dia vão ser úteis"	A regra adotada é: 1 consumidor mantém a lógica local; 2 consumidores, considera extrair; só com 3+ consumidores reais vale extrair um módulo compartilhado
Engine de regras de negócio genérica	Ver caso concreto acima — resolve um problema que a empresa ainda não tem, ao custo de meses de trabalho sem cliente novo
Dashboard gigante antes de ter dado pra mostrar	Fase 4 do roadmap é justamente isso — deliberadamente adiada até haver volume real
Microserviços	O projeto inteiro roda num container de backend + um Postgres — dividir isso em serviços separados não resolve nenhum problema real de escala que exista hoje
Frameworks sem necessidade (React, Next.js, etc.)	Rejeitado explicitamente ao redesenhar a landing — HTML+Tailwind estático já atende
Abstrações prematuras em geral	O agent_construtor.py está congelado justamente pra evitar isso — funciona, não mexer sem necessidade
Duplicação de dado (preço, config)	Já aconteceu uma vez (preço hardcoded no Hunter) e foi corrigido — o aprendizado é checar isso proativamente em qualquer novo código que leia dado "de negócio"
Acoplamento entre a landing (Vercel) e o backend (VM)	A landing funciona hoje mesmo se a VM cair — qualquer mudança que crie essa dependência nova é uma regressão de resiliência, não uma melhoria
Automatizar venda ou WhatsApp antes da hora	Os dois guardrails mais críticos do projeto — violar qualquer um dos dois tem custo alto (banimento de canal, ou vender mal em escala com script não testado)

Inconsistências entre documentação e código encontradas nesta auditoria

Listadas para conhecimento — não corrigidas automaticamente, por decisão de processo (o skill de auditoria deste projeto nunca corrige, só reporta):

1. `.env` local tem `DATABASE_URL` apontando pra uma VM Oracle (`147.15.73.239`) que a própria documentação de infraestrutura (`infra/zero_trust_deploy.md`) registra como **já desligada** — não é um firewall bloqueando, é uma referência a uma máquina que não existe mais.
2. `CLAUDE.md` e `ROADMAP.md` ainda descrevem o backend como pendente de deploy em Render/Railway — a infraestrutura real (Oracle Zero Trust) já está implantada há tempo e nunca foi refletida de volta nesses dois documentos.
3. 8 arquivos `.md` na raiz do repositório (`AGENT_CONSTRUTOR.md` , `RESUMO_EXECUTIVO.md` , `FASE1.md` , `FASE2.md` , `FASE2_FINALIZADA.md` , `SETUP.md` , `API_DOCS.md` , `AGENTS.md`) descrevem uma versão do produto anterior à reorganização em `backend/` — mencionam `ANTHROPIC_API_KEY` como única IA e um modelo fixo antigo, sem qualquer menção a Postgres, Hunter ou Zero Trust.
4. `AGENTS.md` (raiz) referencia um arquivo (`backend/agents/AGENTS.md`) que não existe — o arquivo real chama `backend/agents/CLAUDE.md` .
5. `ROADMAP.md` lista "15-20 vendas fechadas" como pendência, mas não há em nenhum lugar do repositório um número real de vendas já fechadas até hoje — não dá pra saber quanto falta.
6. `docs/hunter_online_spec.md` se descreve como "não implementado", mas parte do que descreve como evolução futura (o pipeline de status manual) **já está implementado e em produção**.